МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №4.2
по дисциплине
«Програмирование»
«Разработка графического интерфейса пользователя (GUI) для взаимодействия со структурой данных. Интеграция модулей.»

Выполнил студент гр. ИВТб-1303-06-00 _____/Пересторонин С.И./
Проверил доцент кафедры ЭВМ _____/Баташев П.А./

Киров
2025

Цель работы: Научиться создавать простые графические интерфейсы с использованием библиотеки tkinter, организовывать обработку событий и интегрировать ранее разработанные программные модули в единое приложение. Закрепить понимание работы выбранной структуры данных через её визуальное представление.

1 Задание:

1. Реализовать программу взаимодействия со структурой данных. Язык программирования и вид структуры указан в варианте.

2. Структура данных должна быть реализована на основе динамической памяти.

3. Взаимодействие со структурой данных (создание и очистка структуры; вставка, чтение и удаление элемента; демонстрация всех элементов структуры с отображением их количества) должно происходить с помощью меню, навигация в котором происходит с помощью стрелок.

4. Так же меню должно содержать одну дополнительную возможность, указанную в варианте. При реализации дополнительной возможности прямой доступ к элементам структуры недопустим, разрешено взаимодействие со структурой только с использованием реализованных подпрограмм в пункте 3.

Структура: Циклическая очередь.
Дополнительная возможность: Поместить максимальное/минимальное значение в начало/середину/конец структуры (тип и место указывает пользователь).

2 Решение:

Схема алгоритма задания представлена на Рисунках 1 - 14.
Исходный код представлен в Приложении А1.
Скриншоты работы программы представлены в Приложение А2.

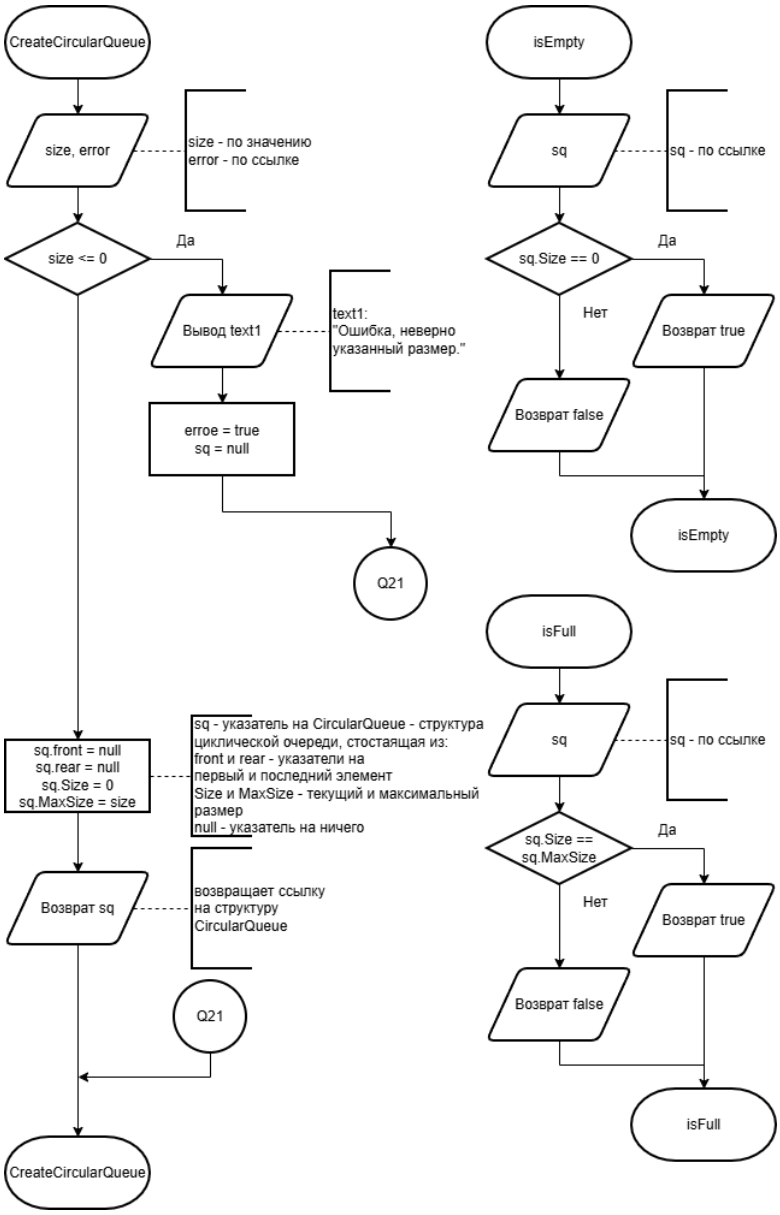


Рисунок 1 - Схема алгоритма задания.

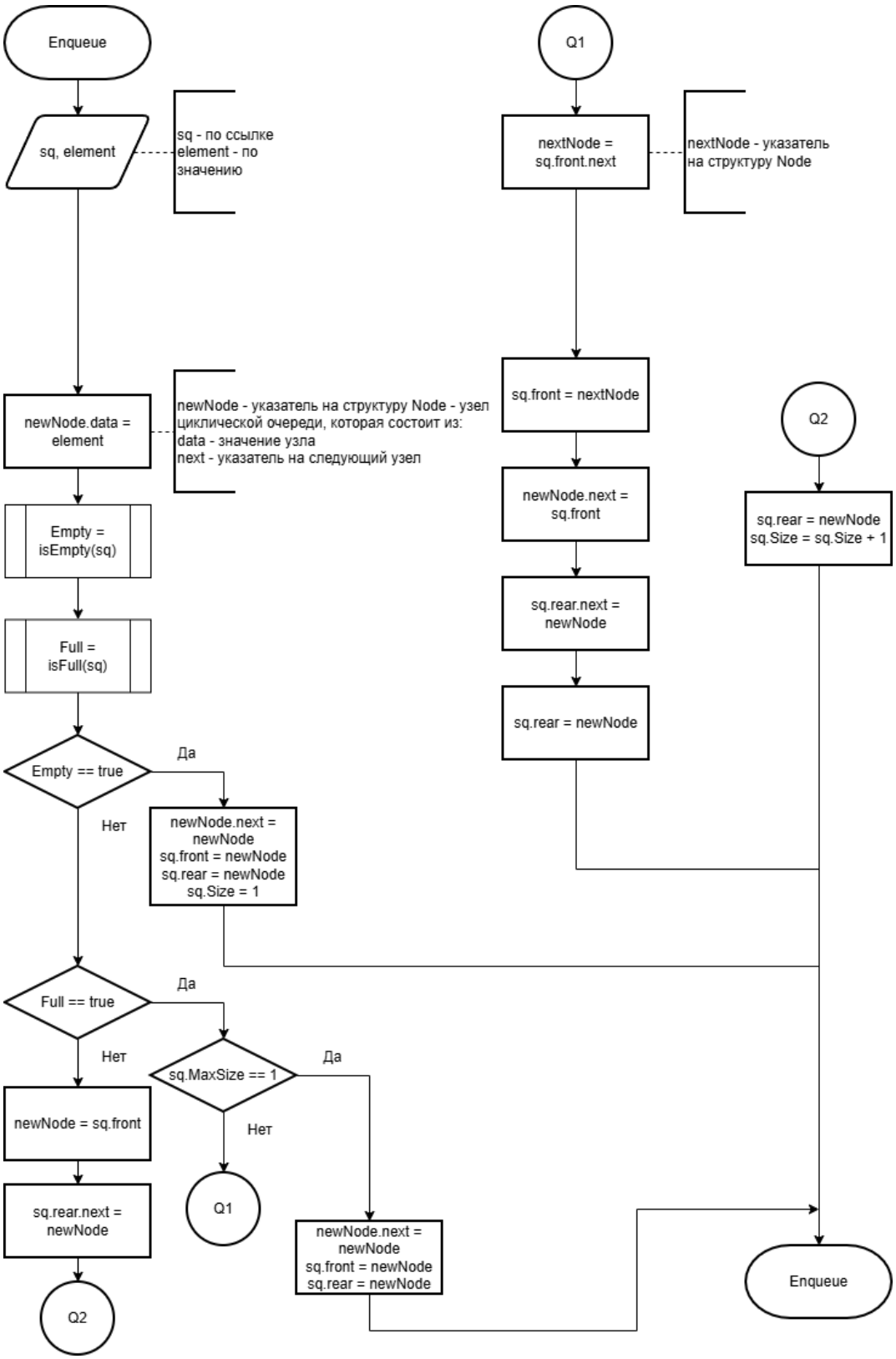


Рисунок 2 - Схема алгоритма задания.

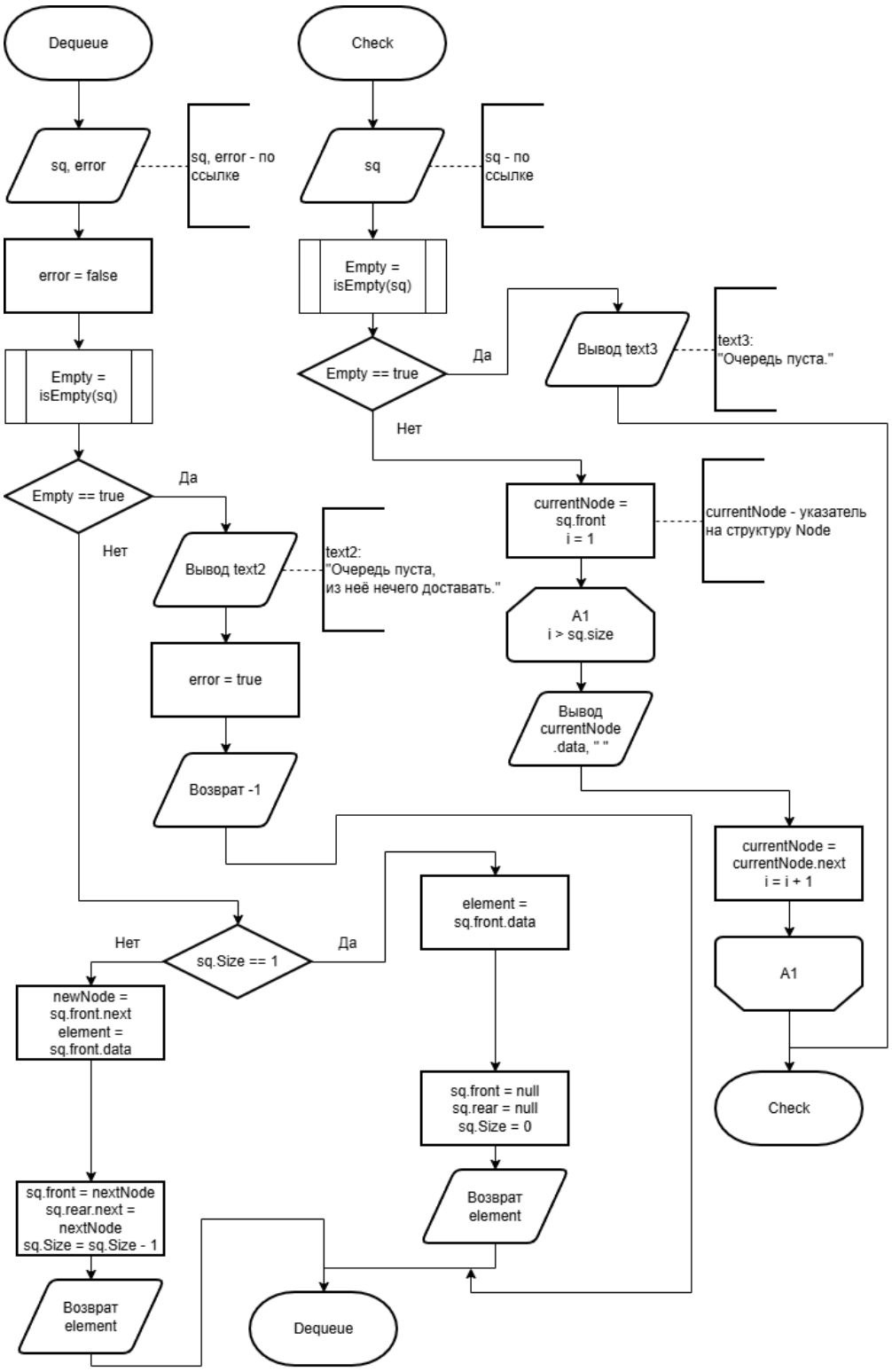


Рисунок 3 - Схема алгоритма задания.

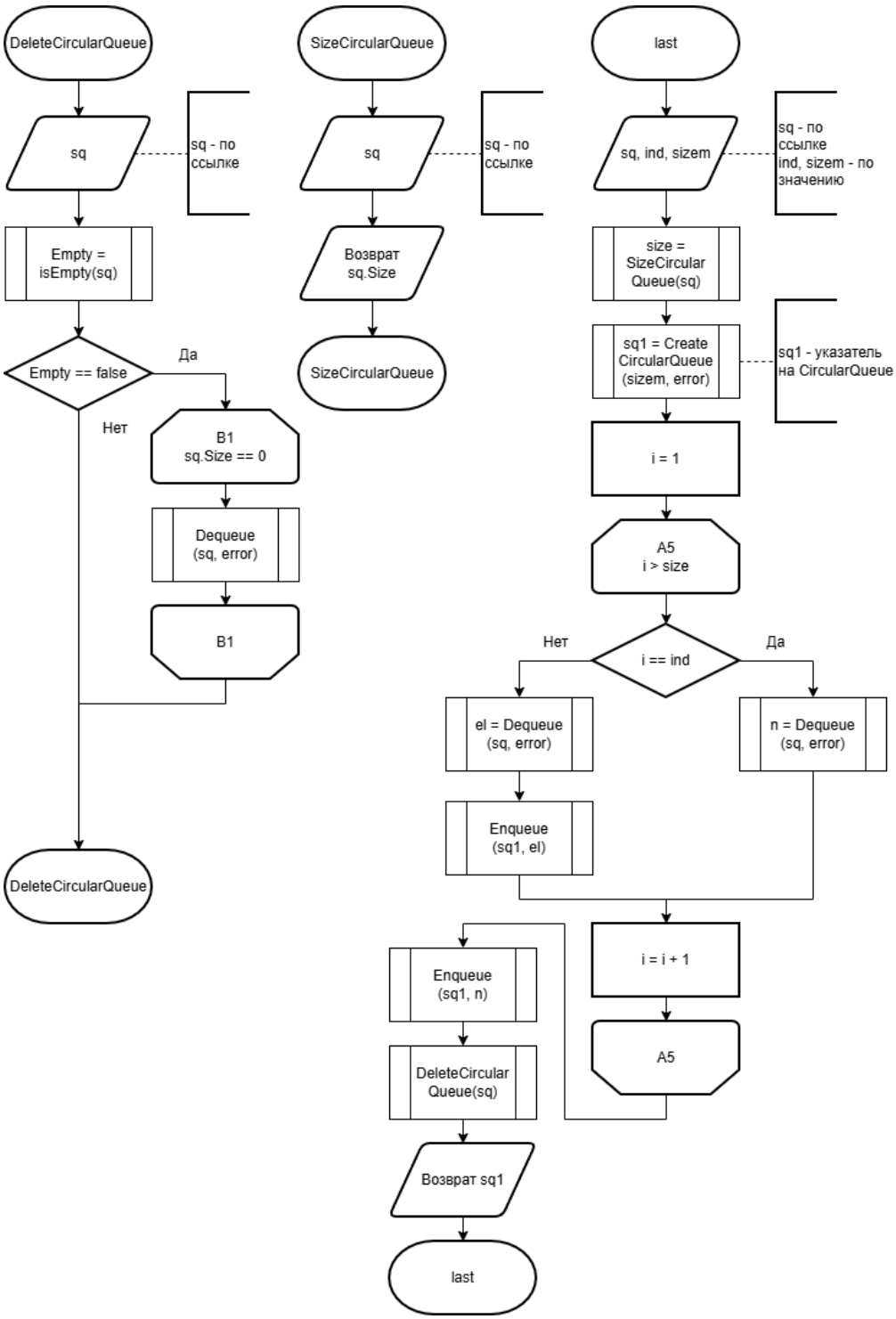


Рисунок 4 - Схема алгоритма задания.

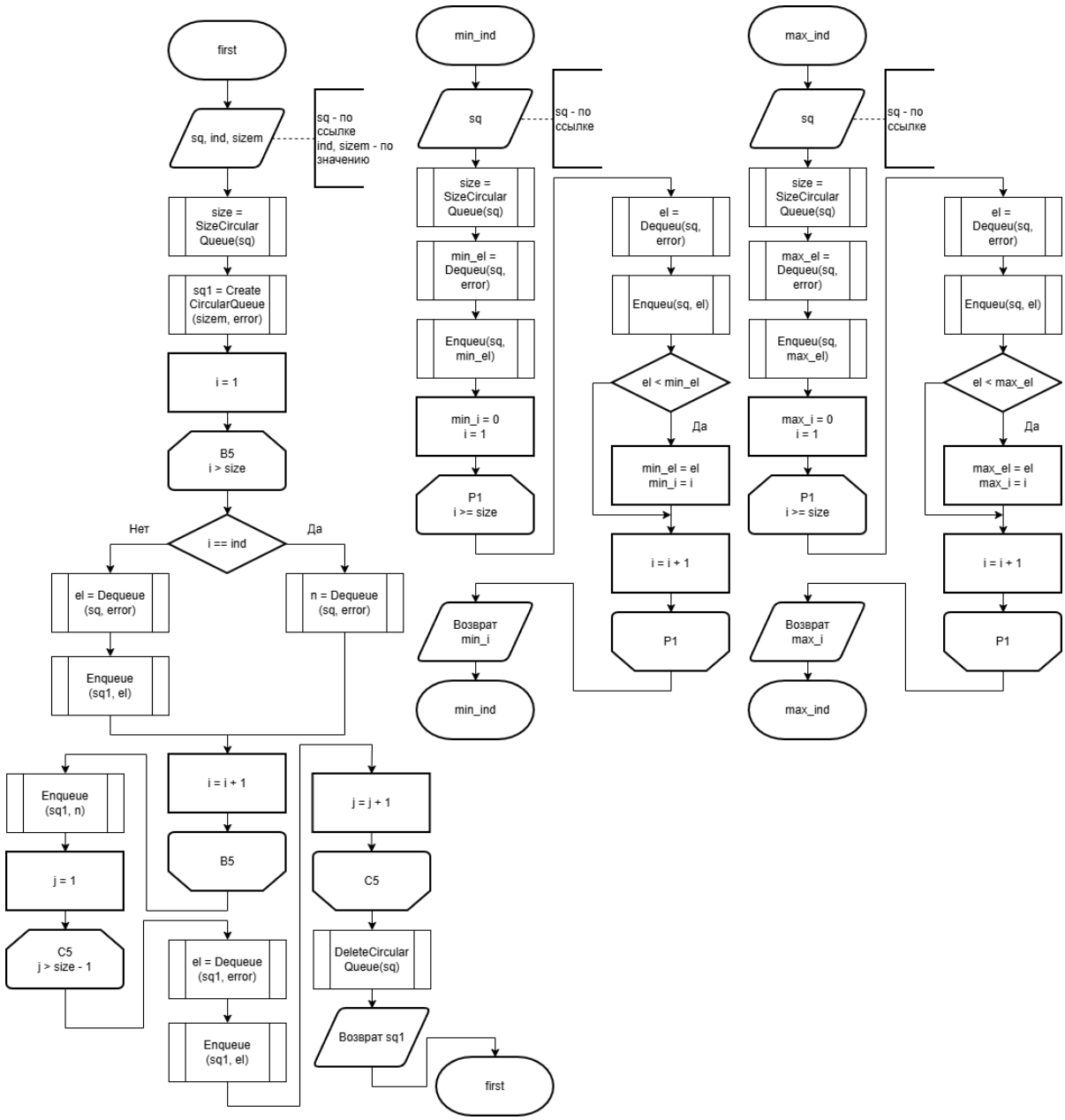


Рисунок 5 - Схема алгоритма задания.

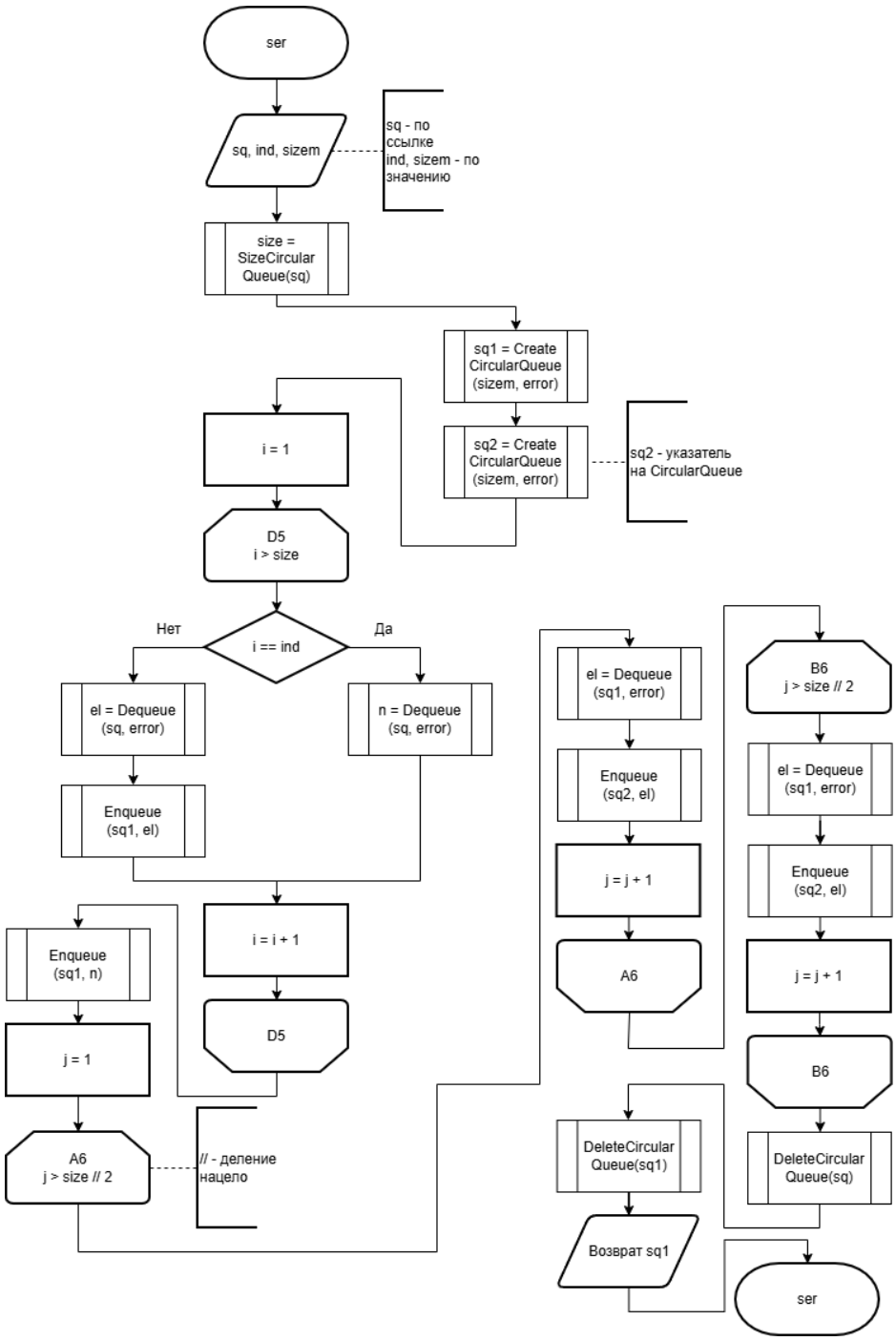


Рисунок 6 - Схема алгоритма задания.

Вывод:

В ходе выполнения лабораторной работы были изучены структуры данных и способы взаимодействия с ними путём использования функций и процедур самой структуры данных, реализация их в виде библиотек и модулей, и последующее использования на других языках.

Приложение А1

Исходный код задания:

CircularQueue1.cpp:

```
#include "Circular_Queue.h"

struct Node {
   Node *next;
   int data;
};

struct CircularQueue {
    Node *front;
    Node *rear;
    int Size;
    int MaxSize;
};

CircularQueue* CreateCircularQueue(int size, bool *error) {
    *error = false;
    if (size <= 0) {
        *error = true;
        return nullptr;
    }
    CircularQueue *cq = new CircularQueue;
    cq->front = nullptr;
    cq->rear = nullptr;
    cq->MaxSize = size;
    cq->Size = 0;
    return cq;
}

bool isEmpty(CircularQueue *cq) {
    return (cq->Size == 0);
```

```
}

bool isFull(CircularQueue *cq) {
    return (cq->Size == cq->MaxSize);
}

void Enqueue(CircularQueue *cq, int element) {
    Node *newNode = new Node;
    newNode->data = element;
    if (isEmpty(cq)) {
        newNode->next = newNode;
        cq->front = newNode;
        cq->rear = newNode;
        cq->Size = 1;
    } else if(isFull(cq)) {
        if (cq->MaxSize == 1) {
            delete cq->front;
            newNode->next = newNode;
            cq->rear = newNode;
            cq->front = newNode;
        }
        else {
            Node* nextNode = cq->front->next;
            delete cq->front;
            cq->front = nextNode;
            newNode->next = cq->front;
            cq->rear->next = newNode;
            cq->rear = newNode;
        }
    } else {
        newNode->next = cq->front;
        cq->rear->next = newNode;
        cq->rear = newNode;
        cq->Size = cq->Size + 1;
    }
}
```

```
int Dequeue(CircularQueue *cq, bool *error) {
    *error = false;
    if (isEmpty(cq)) {
        *error = true;
        return -1;
    }
    else if (cq->Size == 1) {
        int element = cq->front->data;
        delete cq->front;
        cq->front = nullptr;
        cq->rear = nullptr;
        cq->Size = 0;
        return element;
    }
    else {
        Node* nextNode = cq->front->next;
        int element = cq->front->data;
        delete cq->front;
        cq->front = nextNode;
        cq->rear->next = nextNode;
        cq->Size = cq->Size - 1;
        return element;
    }
}

void check(CircularQueue *cq, bool *Empty, int *item) {
    *Empty = false;
    if (isEmpty(cq)) {
        *Empty = true;
    }
    else {
        Node* currentNode = cq->front;
        for (int i = 0; i < cq->Size; i++) {
            item[i] = currentNode->data;
            currentNode = currentNode->next;
        }
    }
```

```
}

void DeleteCircularQueue(CircularQueue *cq) {
    if (cq != nullptr) {
        bool error;
        while (cq->Size > 0) {
            Dequeue(cq, &error);
        }
        delete cq;
    }
}

int SizeCircularQueue(CircularQueue *cq) {
    return cq->Size;
}
```

CircularQueue2.cpp:

```
#include <queue>
#include "Circular_Queue.h"

using namespace std;

struct CircularQueue {
    queue<int> Circular_Queue;
    int MaxSize;
};

CircularQueue* CreateCircularQueue(int size, bool *error) {
    *error = false;
    if (size <= 0) {
        *error = true;
        return nullptr;
    }
    CircularQueue *cq = new CircularQueue;
    cq->MaxSize = size;
    return cq;
}
```

```
bool isEmpty(CircularQueue *cq) {
    return (cq->Circular_Queue.empty() == true);
}

bool isFull(CircularQueue *cq) {
    return (static_cast<int>(cq->Circular_Queue.size()) == cq->MaxSize);
}

void Enqueue(CircularQueue *cq, int element) {
    if(isFull(cq)) {
        cq->Circular_Queue.pop();
        cq->Circular_Queue.push(element);
    } else {
        cq->Circular_Queue.push(element);
    }
}

int Dequeue(CircularQueue *cq, bool *error) {
    *error = false;
    if (isEmpty(cq)) {
        *error = true;
        return -1;
    }
    int element = cq->Circular_Queue.front();
    cq->Circular_Queue.pop();
    return element;
}

void check(CircularQueue *cq, bool *Empty, int *item) {
    *Empty = false;
    if (isEmpty(cq)) {
        *Empty = true;
    }
    else {
        queue<int> qqq = cq->Circular_Queue;
        for (int i = 0; i < cq->Circular_Queue.size(); i++) {
```

```
            item[i] = qqq.front();
            qqq.pop();
        }
    }
}

void DeleteCircularQueue(CircularQueue *cq) {
    if (cq != nullptr) {
        delete cq;
    }
}

int SizeCircularQueue(CircularQueue *cq) {
    return static_cast<int>(cq->Circular_Queue.size());
}
```

CircularQueue.h:

```
#pragma once

extern "C" {
    typedef struct CircularQueue CircularQueue;
    __declspec(dllexport) CircularQueue* CreateCircularQueue(int size, bool *error);
    __declspec(dllexport) bool isEmpty(CircularQueue *cq);
    __declspec(dllexport) bool isFull(CircularQueue *cq);
    __declspec(dllexport) void Enqueue(CircularQueue *cq, int element);
    __declspec(dllexport) int Dequeue(CircularQueue *cq, bool *error);
    __declspec(dllexport) void check(CircularQueue *cq, bool *Empty, int *item);
    __declspec(dllexport) void DeleteCircularQueue(CircularQueue *cq);
    __declspec(dllexport) int SizeCircularQueue(CircularQueue *cq);
}
```

CircularQueue3.py:

```
class Node:
    def __init__(node, data:int):
        node.data = data
        node.next = None
```

```python
class CircularQueue:
    def __init__(circularqueue, MaxSize:int):
        circularqueue.rear = None
        circularqueue.front = None
        circularqueue.MaxSize = MaxSize
        circularqueue.Size = 0

def CreateCircularQueuePY(size:int):
    error = False
    if size <= 0:
        error = True, error
        return None
    return CircularQueue(size), error

def is_emptyPY(cq):
    return cq.Size == 0

def is_fullPY(cq):
    return cq.Size == cq.MaxSize

def EnqueuePY(cq, element:int):
    newNode = Node(element)
    if is_emptyPY(cq):
        newNode.next = newNode
        cq.front = newNode
        cq.rear = newNode
        cq.Size = 1
    elif is_fullPY(cq):
        if cq.MaxSize == 1:
            cq.front = newNode
            cq.rear = newNode
            newNode.next = newNode
        else:
            nextNode = cq.front.next
            cq.front = nextNode
            newNode.next = cq.front
```

```
            cq.rear.next = newNode
            cq.rear = newNode
    else:
        newNode.next = cq.front
        cq.rear.next = newNode
        cq.rear = newNode
        cq.Size += 1

def DequeuePY(cq):
    error = False
    if is_emptyPY(cq):
        error = True
        return -1, error
    if cq.Size == 1:
        element = cq.front.data
        cq.front = None
        cq.rear = None
        cq.Size = 0
        return element, error
    else:
        nextNode = cq.front.next
        element = cq.front.data
        cq.front = nextNode
        cq.rear.next = nextNode
        cq.Size -= 1
        return element, error

def checkPY(cq):
    empty = False
    item = []
    if is_emptyPY(cq):
        empty = True
        return item, empty
    currentNode = cq.front
    for i in range(cq.Size):
        item.append(currentNode.data)
        currentNode = currentNode.next
```

```
    return item, empty

def DeleteCircularQueuePY(cq):
    if cq is not None:
        while cq.Size > 0:
            DequeuePY(cq)

def SizeCircularQueuePY(cq):
    return cq.Size
```

main.py:

```
from ctypes import *
from Circular_Queue3 import *
import tkinter as tk
from tkinter import ttk
from tkinter.scrolledtext import ScrolledText

lib1 = r"./Circular_Queue1.dll"
lib2 = r"./Circular_Queue2.dll"

v1 = False
v2 = False
v3 = False
cq = None
cppLib = None
sizeQueue = 0
index = None

class Node(Structure):
        pass
Node._fields_ = [("next", POINTER(Node)), ("data", c_int)]

class CircularQueue(Structure):
    _fields_ = [("front", POINTER(Node)), ("rear", POINTER(Node)), ("Size", c_int), ("

class libCPP:
    def __init__(self, lib):
```

```
        self.lib = lib
        self.lib = CDLL(lib)
        self.lib.CreateCircularQueue.argtypes = [c_int, POINTER(c_bool)]
        self.lib.CreateCircularQueue.restype = POINTER(CircularQueue)
        self.lib.isEmpty.argtypes = [POINTER(CircularQueue)]
        self.lib.isEmpty.restype = c_bool
        self.lib.isFull.argtypes = [POINTER(CircularQueue)]
        self.lib.isFull.restype = c_bool
        self.lib.Enqueue.argtypes = [POINTER(CircularQueue), c_int]
        self.lib.Enqueue.restype = None
        self.lib.Dequeue.argtypes = [POINTER(CircularQueue), POINTER(c_bool)]
        self.lib.Dequeue.restype = c_int
        self.lib.check.argtypes = [POINTER(CircularQueue), POINTER(c_bool), POINTER(c_
        self.lib.check.restype = None
        self.lib.DeleteCircularQueue.argtypes = [POINTER(CircularQueue)]
        self.lib.DeleteCircularQueue.restype = None
        self.lib.SizeCircularQueue.argtypes = [POINTER(CircularQueue)]
        self.lib.SizeCircularQueue.restype = c_int

def CreateCircularQueue(size:int):
    cq = None
    error = False
    if v3:
        cq, error = CreateCircularQueuePY(size)
    if v1 or v2:
        error = c_bool(False)
        cq = cppLib.lib.CreateCircularQueue(size, byref(error))
    return cq, error

def Enqueue(cq, element:int):
    if v3:
        EnqueuePY(cq, element)
    if v1 or v2:
        cppLib.lib.Enqueue(cq, element)

def isEmpty(cq):
    if v3:
```

```
        return is_emptyPY(cq)
    if v1 or v2:
        return cppLib.lib.isEmpty(cq)

def isFull(cq):
    if v3:
        return is_fullPY(cq)
    if v1 or v2:
        return cppLib.lib.isFull(cq)

def Dequeue(cq):
    if v3:
        el, error = DequeuePY(cq)
    if v1 or v2:
        error = c_bool(False)
        el = cppLib.lib.Dequeue(cq, byref(error))
    return el, error

def SizeCircularQueue(cq):
    if v3:
        return SizeCircularQueuePY(cq)
    if v1 or v2:
        return cppLib.lib.SizeCircularQueue(cq)

def checkButton():
    if v3:
        a, empty = checkPY(cq)
    if v1 or v2:
        empty = c_bool(False)
        a = (c_int * SizeCircularQueue(cq))()
        cppLib.lib.check(cq, byref(empty), a)
    if empty:
        output.insert(tk.END, "Очередь пуста.\n")
        output.see(tk.END)
    else:
        output.insert(tk.END, "Элементы очереди: " + str(list(a)) + "\n")
        output.see(tk.END)
```

```python
def DeleteCircularQueue(cq):
    if v3:
        DeleteCircularQueuePY(cq)
    if v1 or v2:
        cppLib.lib.DeleteCircularQueue(cq)

def min_ind(cq):
    size = SizeCircularQueue(cq)
    min_el, error = Dequeue(cq)
    Enqueue(cq, min_el)
    min_i = 0
    for i in range(1, size):
        el1, error = Dequeue(cq)
        Enqueue(cq, el1)
        if el1 < min_el:
            min_el = el1
            min_i = i
    return min_i

def max_ind(cq):
    size = SizeCircularQueue(cq)
    max_el, error = Dequeue(cq)
    Enqueue(cq, max_el)
    max_i = 0
    for i in range(1, size):
        el1, error = Dequeue(cq)
        Enqueue(cq, el1)
        if el1 > max_el:
            max_el = el1
            max_i = i
    return max_i

def last(cq, ind:int, MaxSize:int):
    size = SizeCircularQueue(cq)
    cq1, error = CreateCircularQueue(MaxSize)
    for i in range(size):
```

```
        if i == ind:
            n, error = Dequeue(cq)
        else:
            el, error = Dequeue(cq)
            Enqueue(cq1, el)
    Enqueue(cq1, n)
    DeleteCircularQueue(cq)
    return cq1

def first(cq, ind:int, MaxSize:int):
    size = SizeCircularQueue(cq)
    cq1, error = CreateCircularQueue(MaxSize)
    for i in range(size):
        if i == ind:
            n, error = Dequeue(cq)
        else:
            el, error = Dequeue(cq)
            Enqueue(cq1, el)
    Enqueue(cq1, n)
    for i in range(size - 1):
        el, error = Dequeue(cq1)
        Enqueue(cq1, el)
    DeleteCircularQueue(cq)
    return cq1

def ser(cq, ind:int, MaxSize:int):
    size = SizeCircularQueue(cq)
    cq1, error = CreateCircularQueue(MaxSize)
    cq2, error = CreateCircularQueue(MaxSize)
    for i in range(size):
        if i == ind:
            n, error = Dequeue(cq)
        else:
            el, error = Dequeue(cq)
            Enqueue(cq1, el)
    for i in range(size // 2):
        el, error = Dequeue(cq1)
```

```
        Enqueue(cq2, el)
    Enqueue(cq2, n)
    for i in range(size // 2):
        el, error = Dequeue(cq1)
        Enqueue(cq2, el)
    DeleteCircularQueue(cq)
    DeleteCircularQueue(cq1)
    return cq2

def SetLib():
    global v1, v2, v3, cppLib
    lib = lib_choice.get()
    if lib == "C++ динамическая":
        v1 = True
        v2 = False
        v3 = False
        cppLib = libCPP(lib1)
    elif lib == "C++ STL":
        v1 = False
        v2 = True
        v3 = False
        cppLib = libCPP(lib2)
    elif lib == "Python":
        v1 = False
        v2 = False
        v3 = True
        cppLib = None
    output.insert(tk.END, "Выбрвна: " + lib + "\n")
    output.see(tk.END)
    lib_frame.place_forget()
    len_frame.place(relx=0.5, rely=0.4, anchor=tk.CENTER)

def BackLib():
    global v1, v2, v3
    v1 = False
    v2 = False
    v3 = False
```

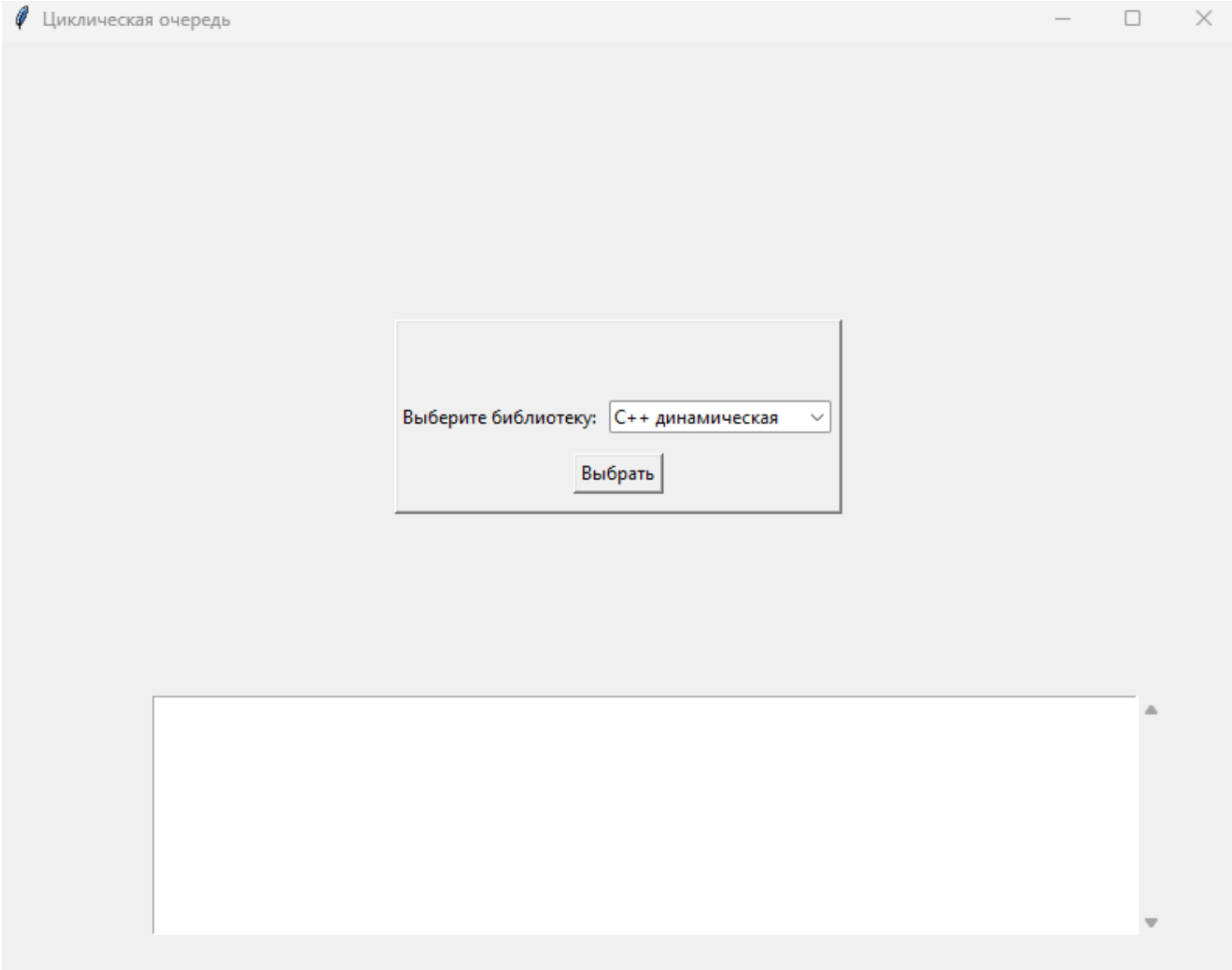
```
    lib_frame.place(relx=0.5, rely=0.4, anchor=tk.CENTER)
    len_frame.place_forget()

def Create():
    global cq, sizeQueue
    try:
        try:
            sizeQueue = int(entry1.get())
        except:
            raise ValueError("Не удалось преобразовать введенное значение в число")
        if sizeQueue < 0:
            raise ValueError("Значение не может быть меньше 0")
        entry1.delete(0, tk.END)
        len_frame.place_forget()
        main_frame.place(relx=0.5, rely=0.4, anchor=tk.CENTER)
        cq, error = CreateCircularQueue(sizeQueue)
    except Exception as e:
        output.insert(tk.END, f"Ошибка, неверно указанный размер (Подробнее: {e}).\n")
        output.see(tk.END)
        entry1.delete(0, tk.END)

def EnqueueButton():
    if entry2.get() == "":
        return
    el = int(entry2.get())
    entry2.delete(0, tk.END)
    Enqueue(cq, el)
    output.insert(tk.END, "Добвавлен элемент: " + str(el) + "\n")
    output.see(tk.END)

def DequeueButton():
    el, error = Dequeue(cq)
    if error:
        output.insert(tk.END, "Очередь пуста из неё ничего нельзя удалить.\n")
        output.see(tk.END)
        return
    output.insert(tk.END, "Удалён элемент: " + str(el) + "\n")
```

```
    output.see(tk.END)

def DeleteCircularQueueButton():
    global sizeQueue
    DeleteCircularQueue(cq)
    main_frame.place_forget()
    len_frame.place(relx=0.5, rely=0.4, anchor=tk.CENTER)
    sizeQueue = 0

def SizeButton():
    output.insert(tk.END, "Текущий размер оереди: " + str(SizeCircularQueue(cq)) + "\n
    output.see(tk.END)

def dopzad():
    if (SizeCircularQueue(cq) <= 0):
        output.insert(tk.END, "Очередь пуста\n")
        output.see(tk.END)
    else:
        main_frame.place_forget()
        dop1_frame.place(relx=0.5, rely=0.4, anchor=tk.CENTER)

def MinButton():
    global index
    index = min_ind(cq)
    dop1_frame.place_forget()
    dop2_frame.place(relx=0.5, rely=0.4, anchor=tk.CENTER)

def MaxButton():
    global index
    index = max_ind(cq)
    dop1_frame.place_forget()
    dop2_frame.place(relx=0.5, rely=0.4, anchor=tk.CENTER)

def firstButton():
    global cq, index
    cq = first(cq, index, sizeQueue)
    index = None
```

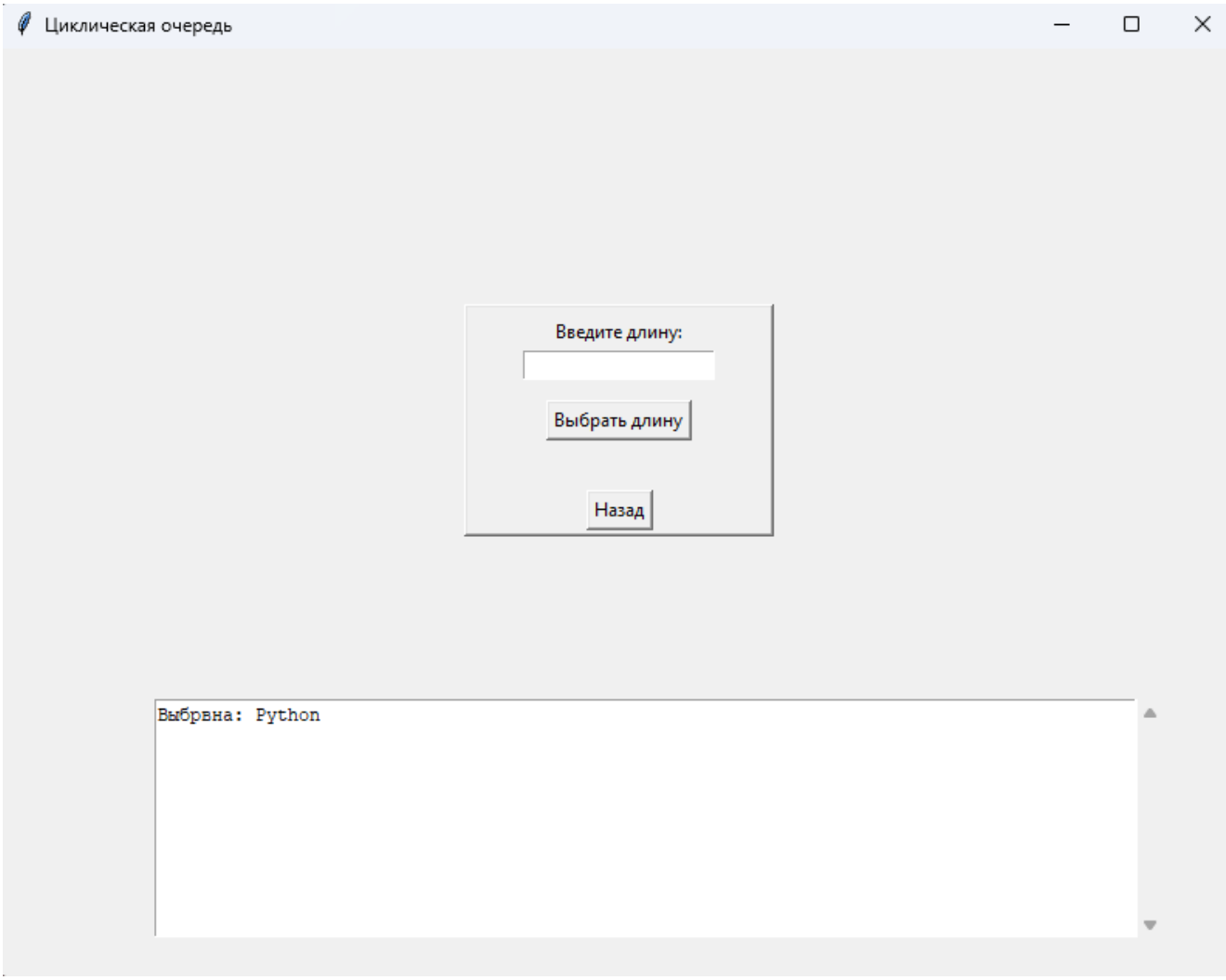
```
    dop2_frame.place_forget()
    main_frame.place(relx=0.5, rely=0.4, anchor=tk.CENTER)

def serButton():
    global cq, index
    if SizeCircularQueue(cq) % 2 == 1:
        cq = ser(cq, index, sizeQueue)
        index = None
        dop2_frame.place_forget()
        main_frame.place(relx=0.5, rely=0.4, anchor=tk.CENTER)
    else:
        output.insert(tk.END, "Колличество элементов чётное, середины нет.\n")
        output.see(tk.END)

def lastButton():
    global cq, index
    cq = last(cq, index, sizeQueue)
    index = None
    dop2_frame.place_forget()
    main_frame.place(relx=0.5, rely=0.4, anchor=tk.CENTER)

root = tk.Tk()
root.geometry("800x600")
root.title("Циклическая очередь")

lib_frame = tk.Frame(root, relief=tk.RAISED, bd=2)
lib_frame.place(relx=0.5, rely=0.4, anchor=tk.CENTER)
tk.Label(lib_frame, text="Выберите библиотеку:" ).pack(side=tk.LEFT, padx=0, pady=50)
tk.Button(lib_frame, text = "Выбрать", command = SetLib).place(relx=0.5, rely=0.8, anc
lib_choice = tk.StringVar()
lib_combo = ttk.Combobox(lib_frame, textvariable=lib_choice, values=["C++ динамическая
lib_combo.pack(side=tk.LEFT, padx=5, pady=5)
lib_combo.current(0)

output = ScrolledText(root, width=90, height=10, font=("Courier New", 9))
output.pack(pady=150, padx=150, fill=tk.BOTH, expand=True)
output.place(x=100, y=420)
```

```
len_frame = tk.Frame(root, width=200, height=150, bd=2, relief=tk.RAISED)
entry1 = tk.Entry(len_frame)
entry1.place(relx=0.5, rely=0.25, anchor=tk.CENTER)
tk.Label(len_frame, text="Введите длину:" ).place(relx=0.5, rely=0.1, anchor=tk.CENTER
tk.Button(len_frame, text = "Выбрать длину", command = Create).place(relx=0.5, rely=0.
tk.Button(len_frame, text = "Назад", command = BackLib).place(relx=0.5, rely=0.9, anch

main_frame = tk.Frame(root, width=400, height=300, bd=2, relief=tk.RAISED)
entry2 = tk.Entry(main_frame)
entry2.place(relx=0.5, rely=0.1, anchor=tk.CENTER)
tk.Button(main_frame, text = "Добавить элемент", command = EnqueueButton).place(relx=0
tk.Button(main_frame, text = "Удалить элемент", command = DequeueButton).place(relx=0.
tk.Button(main_frame, text = "Размер очереди", command = SizeButton).place(relx=0.5, r
tk.Button(main_frame, text = "Посмотреть элементы очереди", command = checkButton).pla
tk.Button(main_frame, text = "Дополнительное задание", command = dopzad).place(relx=0.
tk.Button(main_frame, text = "Удалить очередь", command = DeleteCircularQueueButton).p

dop1_frame = tk.Frame(root, width=150, height=100, bd=2, relief=tk.RAISED)
tk.Button(dop1_frame, text = "Минимальный", command = MinButton).place(relx=0.5, rely=
tk.Button(dop1_frame, text = "Максимальный", command = MaxButton).place(relx=0.5, rely

dop2_frame = tk.Frame(root, width=150, height=150, bd=2, relief=tk.RAISED)
tk.Button(dop2_frame, text = "Начало", command = firstButton).place(relx=0.5, rely=0.3
tk.Button(dop2_frame, text = "Середина", command = serButton).place(relx=0.5, rely=0.5
tk.Button(dop2_frame, text = "Конец", command = lastButton).place(relx=0.5, rely=0.7,

root.mainloop()
```

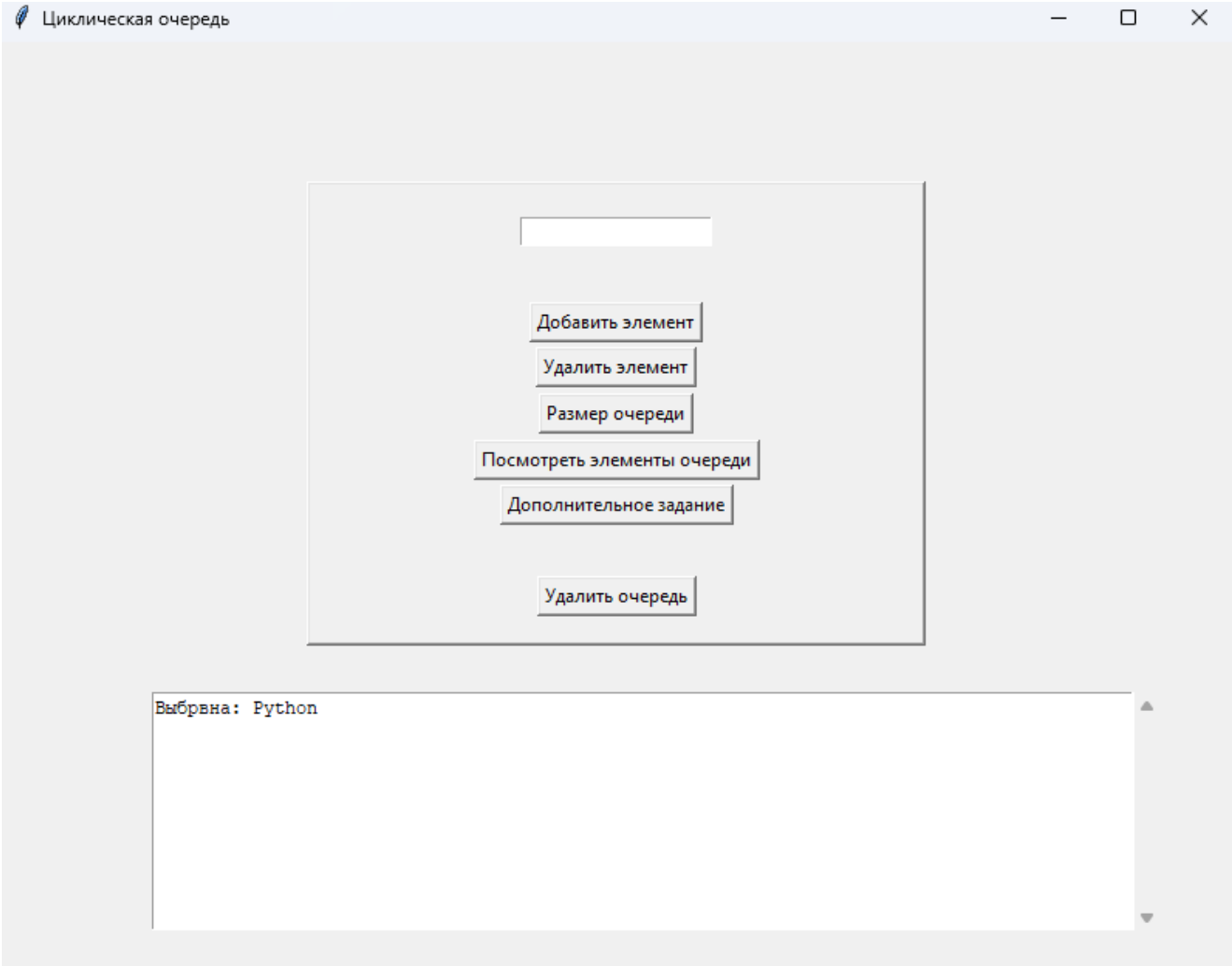
Приложение А2



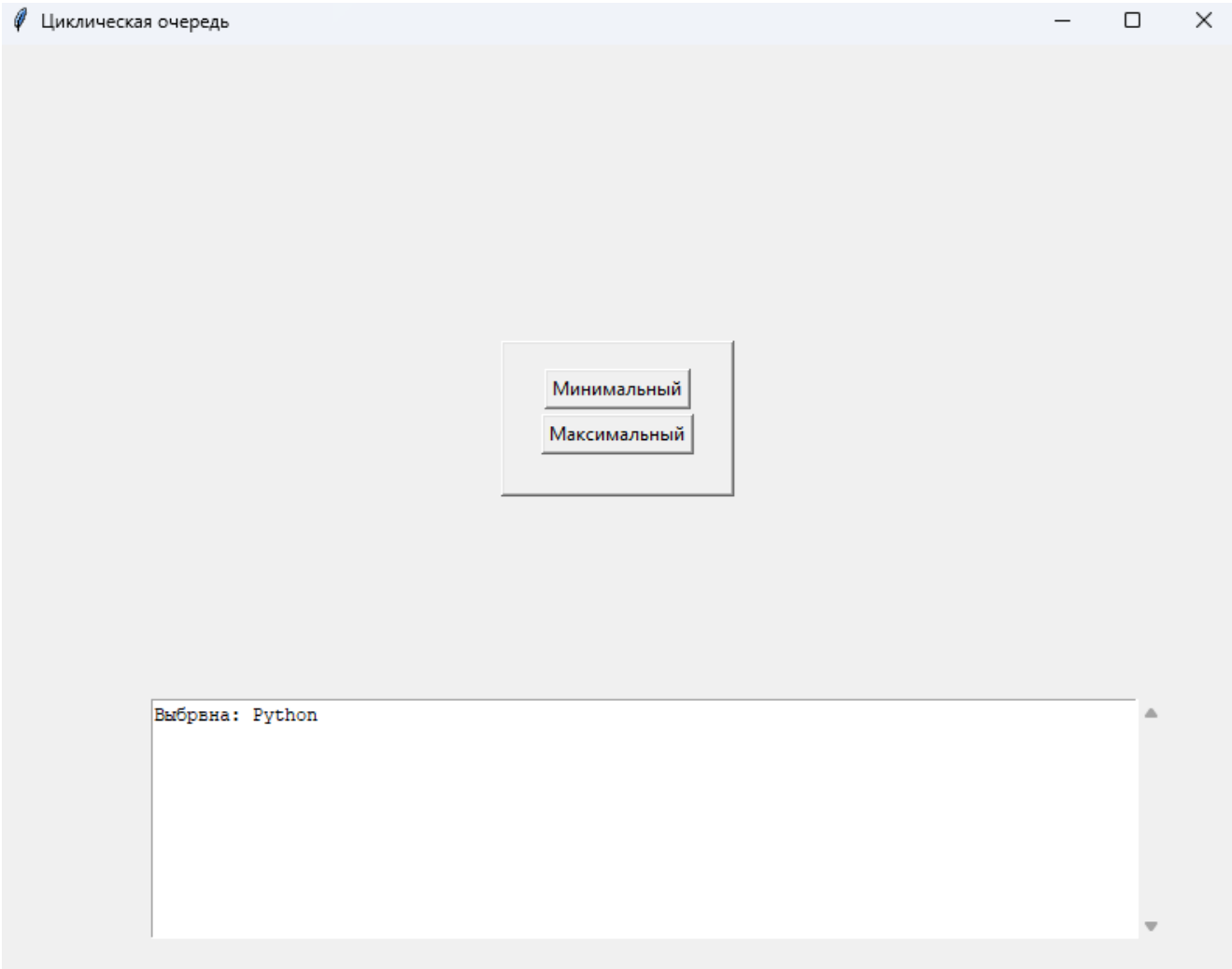
Скриншот 1 - Выбор библиотеки.



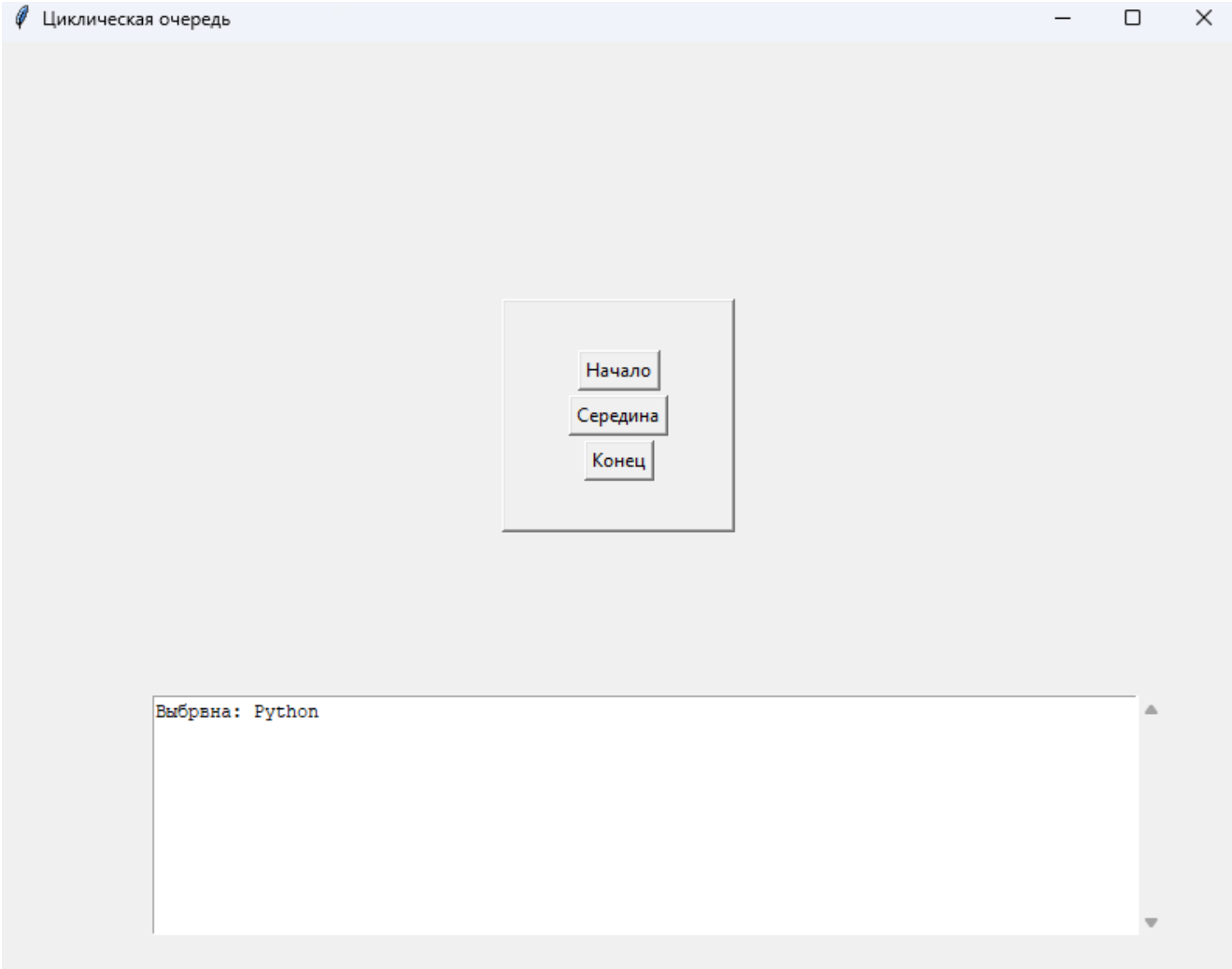
Скриншот 2 - Создание циклической очереди.



Скриншот 3 - Меню циклической очереди.



Скриншот 4 - Дополнительное задани.



Скриншот 5 - Дополнительное задани.